技术报告

实验鼠目标检测：双 GPU 训练与 iOS 部署下的轻量目标检测系统研究

上海交通大学医学院
网络信息中心 · 深度学习实习项目

PaddleDetection 2.6 · 双 Tesla T4 · React Native iOS

2026 年 3 月

摘要

本文基于 PaddleDetection 2.6 框架，在双 Tesla T4 GPU 服务器上对实验鼠目标检测进行了系统研究。我们从三个异构数据源构建了 7,286 张图像的 VOC 格式数据集（5,828 训练 / 1,458 验证），涵盖两个类别：*mouse*（实验鼠）与 *other*（其他）。我们设计了 4×2 的对照实验矩阵，覆盖两种模型族（PicoDet-S 与 YOLOv3-MobileNetV1）、两种 GPU 配置（单卡 / 双卡）及两种数据规模（全量 / 三分之一子集），共八组训练运行，累计约 270 GPU-小时。

学习率依据线性缩放规则（$\eta = \eta_{\text{base}} \times B/B_{\text{ref}}$）确定，以将批量大小的影响与并行化效应分离。本文最优模型为 PicoDet-S 单卡小批量训练（batch = 32, lr = 0.08），在 epoch 70 时达到 **mAP@0.5 = 94.30%**，较 YOLOv3 最佳结果（91.54%）高出 **2.76 个百分点**，同时模型体积 **缩小 21 倍**（4.4 MB vs. 92 MB），在相同 GPU 上推理速度 **提升 2.2 倍**。一个反直觉的发现是：小批量单卡训练优于标准双卡训练（+0.5 pp），我们将其归因于梯度噪声在 5,828 样本规模数据集上起到的隐式正则化作用。

训练后的 PicoDet-S 模型被导出为 ONNX 格式，集成至基于 React Native 的应用（ONNX Runtime 与 CoreML 后端），并部署于 iPhone。经过四轮工程迭代，推理速度由 1 FPS 提升至稳定 **14 FPS**（热机状态）。我们还研究了数据规模对 YOLOv3 的显著影响：将训练集扩大三倍（1,942 → 5,828 张）使 mAP 提高 **46.9 个百分点**（44.4% → 91.3%），证实在小领域数据集上，数据工程是锚框式重型架构精度的主要驱动力。

关键词： 目标检测、PicoDet-S、YOLOv3、迁移学习、线性缩放规则、移动端部署、ONNX Runtime、iOS。

目录

1 引言 **4**

2 数据集与数据工程 **4**
- 2.1 数据来源 4
- 2.2 预处理与标注统一 5
- 2.3 数据集统计 5
- 2.4 用于消融实验的三分之一子集 5

3 模型架构 **6**
- 3.1 PicoDet-S 6
- 3.2 YOLOv3-MobileNetV1 6
- 3.3 架构对比 7

4 实验设计 **7**
- 4.1 线性缩放规则 7
- 4.2 实验矩阵 8
  - 4.2.1 PicoDet-S（L 系列，全量数据） 8
  - 4.2.2 YOLOv3（Y 系列，可变数据规模） 8
- 4.3 基础设施 8
- 4.4 评估协议 8

5 结果与分析 **9**
- 5.1 八组运行结果汇总 9
- 5.2 PicoDet-S：学习曲线与过拟合 9
- 5.3 PicoDet-S 消融：批量大小、GPU 数量与训练时长 10
- 5.4 YOLOv3：数据规模效应 11
- 5.5 YOLOv3 收敛曲线 11
- 5.6 跨模型对比：精度—效率权衡 12
- 5.7 训练效率 13

6 移动端部署 **13**
- 6.1 导出流程 13
- 6.2 YOLOv3 部署难点 14
- 6.3 迭代优化至 14 FPS 14

7 后续工作 **15**
- 7.1 A 组：模型缩放 15
- 7.2 B 组：YOLOv3 收敛改进 15
- 7.3 C 组：压缩流程 15

7.4 推荐优先级 16

8 结论 **16**

1 引言

实验室动物行为的自动分析已成为生物医学研究的重要工具。在多样化笼养环境中可靠地定位实验鼠——包括杂乱背景、部分遮挡及光照变化——是轨迹追踪、姿态估计等下游行为量化任务的前提。

经典的光帧差分或背景减除方法在光照变化或多动物交互时性能下降。现代深度学习目标检测器具有更强的鲁棒性，但其实际部署涉及本文着力解决的两个实际问题：

1. **训练效率：**在有限 GPU 预算（双 Tesla T4，无 InfiniBand）下，批量大小、学习率、GPU 数量及数据规模如何共同影响最终精度？
2. **移动端部署：**训练得到的模型能否在设备端（iPhone）以交互帧率运行，而无需专用硬件加速器？

贡献

- 我们构建并发布了一个 7,286 张图像的二分类实验鼠检测数据集，由三个异构数据源整合而成，提供训练/验证划分及 VOC 格式标注。
- 我们设计并执行了 4×2 对照实验矩阵，学习率依据线性缩放规则设定，在八组训练中单独考察 GPU 数量、批量大小、训练时长及数据规模的影响。
- 我们发现并量化了反直觉的精度反转：小批量单卡训练（mAP = 94.30%）优于标准双卡配置（mAP = 93.80%），并通过小数据集上梯度噪声的隐式正则化效应进行解释。
- 我们记录了从 PaddlePaddle 训练、ONNX 导出到 React Native iOS 推理的完整部署流程，记录了将吞吐从 1 FPS 提升至 14 FPS 的四轮工程迭代。
- 我们展示了数据规模对 YOLOv3 的显著影响：训练图像增加 3× 使 mAP 提升 46.9 个百分点（44.4% → 91.3%），为锚框式检测器的“数据优先”原则提供了定量依据。

论文结构

Section 2 描述数据采集与预处理。Section 3 介绍两种模型架构。Section 4 详述实验设计。Section 5 呈现定量结果与分析。Section 6 介绍 iOS 部署流程。Section 7 概述后续工作。Section 8 总结全文。

2 数据集与数据工程

2.1 数据来源

数据集由三个异构来源整合而成：

1. **实验室采集**。于上海交通大学医学院实验笼内拍摄的图像，使用 VOC XML 标注格式进行标注。覆盖多种光照条件与笼舍配置。
2. **开源数据集**。公开的啮齿动物图像集，经重新标注并映射至我们的二分类体系。
3. **网络爬取图像**。通过关键词检索获取的照片，经人工筛选剔除近重复与错误标注样本。

2.2 预处理与标注统一

所有图像统一为 Pascal VOC 格式 [2]，并采用一致的两类标签体系：`mouse`（实验鼠，*Mus musculus*）与 `other`（其他非鼠前景目标）。预处理步骤包括：

- 标签统一：合并同义类别名称（*rat*、*mice*、*rodent* → `mouse`）。
- 近重复剔除：使用感知哈希，汉明距离阈值为 4。
- 边界框质量筛选：剔除宽高比 >10 或面积 $< 400\,\mathrm{px}^2$ 的框。

2.3 数据集统计

Table 1 汇总了预处理后的数据集规模。

表 1: 预处理后的数据集概况。

划分	图像数	占比
训练集（全量）	5,828	79.9%
验证集	1,458	20.1%
总计	**7,286**	100%

2.4 用于消融实验的三分之一子集

为量化数据规模对 YOLOv3 的影响（见 Figure 1），我们通过分层随机抽样脚本（`scripts/prepare_1of3_dataset.py`）生成了包含 1,942 张图像的三分之一训练子集。验证集在全量与子集实验间共享，以保证 mAP 可直接对比。该设计将训练集大小作为 Y1/Y2 组（1,942 张）与 Y3/Y4 组（5,828 张）间的唯一变量。

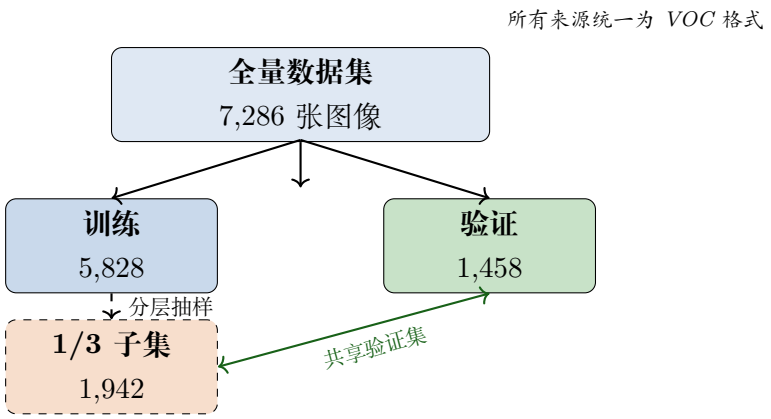

图 1: 数据集划分设计。验证集在全量与三分之一子集实验间共享，以便直接进行 mAP 比较。

3 模型架构

我们研究了两类检测器，代表精度—效率帕累托前沿上的不同取舍。

3.1 PicoDet-S

PicoDet [9] 是一种面向移动端实时推理的无锚单阶段检测器。PicoDet-S（小型）变体采用：

- **骨干网络：** LCNet-0.75× [1]，轻量深度可分离网络，含挤压激励块，在 ImageNet 上预训练。
- **颈部：** LCPAN，轻量路径聚合网络，结合自顶向下与自底向上的特征融合。
- **检测头：** TOOD 风格的任务对齐分类与回归头，采用 Varifocal Loss (VFL) 与 Distribution Focal Loss (DFL)。
- **输入分辨率：** 320×320。
- **参数量：** ≈1.2 M；**模型大小：** 4.4 MB（FP32 ONNX）。

无锚设计在小数据集上尤为有利，因无需针对训练数据分布手工调优锚框配置。骨干与颈部使用 COCO 预训练权重；分类头为两类输出重新初始化。

3.2 YOLOv3-MobileNetV1

YOLOv3 [7] 为基于锚框的多尺度检测器。我们采用 PaddleDetection 官方实现，backbone 为 MobileNetV1 [5]：

- **骨干网络：** 在 COCO 上预训练的 MobileNetV1（80 类头舍弃，特征提取器保留）。
- **检测头：** 三尺度预测（52×52、26×26、13×13），九个默认锚框（源自 COCO）。

- **输入分辨率：** 608×608。
- **参数量：** ≈23 M；**模型大小：** 92 MB（FP32 ONNX）。
- **后处理：** `multiclass_nms3`（PaddleDetection 自定义算子）。

3.3 架构对比

Table 2 对两种架构的关键属性进行了对比。

表 2: 两种检测器架构对比。

属性	**PicoDet-S**	**YOLOv3-MobileNetV1**
检测器类型	无锚框	基于锚框
骨干网络	LCNet-0.75×	MobileNetV1
输入尺寸	320×320	608×608
参数量	≈1.2 M	≈23 M
ONNX 大小 (FP32)	4.4 MB	92 MB
Tesla T4 FPS	∼80	∼35
CoreML 兼容性	✓	×（自定义 NMS 算子）
COCO 预训练	LCNet 骨干 + LCPAN 颈部	MobileNetV1 骨干

4 实验设计

4.1 线性缩放规则

所有学习率均依据 Goyal 等 [3] 的线性缩放规则确定：

$$\eta = \eta_{\text{base}} \times \frac{B_{\text{total}}}{B_{\text{ref}}}, \tag{1}$$

其中 B_{total} 为总有效批量大小（每卡批量 × GPU 数），B_{ref} 为参考批量大小，η_{base} 为对应参考学习率。参考取值为：

- **PicoDet-S：** $\eta_{\text{base}} = 0.08$，$B_{\text{ref}} = 32$（单卡基线）。
- **YOLOv3：** $\eta_{\text{base}} = 0.00125$，$B_{\text{ref}} = 16$（单卡基线，每卡 batch=16）。

所有运行在开始均采用 300 步线性预热，随后为余弦衰减。

4.2 实验矩阵

4.2.1 PicoDet-S（L 系列，全量数据）

四组运行在保持 5,828 张训练图像不变的前提下，变化 GPU 数量、每卡批量大小及训练时长（Table 3）。

表 3: PicoDet-S 实验配置（L 系列）。

运行	GPU	BS/GPU	总 BS	学习率	Epoch	每 Epoch 步数	说明
L1	1	32	32	0.08	300	182	单卡基线
L2	2	32	64	0.16	300	91	双卡标准
L3	2	48	96	0.24	300	61	大批量
L4	2	32	64	0.16	600	91	延长训练

4.2.2 YOLOv3（Y 系列，可变数据规模）

四组运行在 2×2 因子设计中同时变化 GPU 数量与训练集大小（Table 4）。所有运行均采用 80 个 epoch，每卡批量 16。

表 4: YOLOv3 实验配置（Y 系列）。

运行	GPU	总 BS	学习率	训练图像	Epoch	说明
Y1	1	16	0.00125	1,942	80	1/3 数据，单卡
Y2	2	32	0.00250	1,942	80	1/3 数据，双卡
Y3	1	16	0.00125	5,828	80	全量数据，单卡
Y4	2	32	0.00250	5,828	80	全量数据，双卡

4.3 基础设施

所有实验在配备双 NVIDIA Tesla T4（各 15 GB 显存）的服务器上运行。多 GPU 通信通过 TCP 使用 GLOO 后端（因无 InfiniBand 而未使用 NCCL），对当前模型规模开销可忽略。实测双卡加速比为 1.97–2.22×，接近理论 2× 上限。

训练由两个链式执行的 Python 脚本编排（L 系列为 `run_lightweight.py`，Y 系列为 `run_yolov3.py`），依次启动运行，将指标记录至 `output/` 下各组目录，并写入汇总 CSV 文件。

4.4 评估协议

模型精度以 mAP@0.5（IoU 阈值 0.5 下的平均精度均值）衡量，采用 PaddleDetection 评估流程提供的 COCO 风格积分计算。每 10 个 epoch 在共享的 1,458 张验证集上评估

一次；`best_model.pdparams` 在达到新高时更新。FPS 在相同 Tesla T4 服务器上、batch size 1，使用框架内置 profiler 测量。

5 结果与分析

5.1 八组运行结果汇总

表 5: 全部八组训练运行的完整结果。↑ 表示越高越好。

运行	模型	数据	GPU	mAP@0.5 (%)	峰值 Epoch	FPS	时间 (h)
L1	PicoDet-S	全量	1	**94.30**	70	80	14.9
L2	PicoDet-S	全量	2	93.80	∼80	77	6.7
L3	PicoDet-S	全量	2	94.10	∼60	71	6.6
L4	PicoDet-S	全量	2	93.50	∼140	74	13.3
Y1	YOLOv3	1/3	1	44.40	∼65	36	6.7
Y2	YOLOv3	1/3	2	44.10	∼65	36	3.5
Y3	YOLOv3	全量	1	91.30	>80	36	17.4
Y4	YOLOv3	全量	2	**91.54**	>80	35	8.8

5.2 PicoDet-S：学习曲线与过拟合

Figure 2 展示了 L1 运行的每 epoch mAP 曲线。曲线呈现三个阶段：

- **快速上升（epoch 0–70）：** mAP 从 epoch 10 的 91.16% 升至 epoch 70 的全局峰值 94.30%，得益于 COCO 预训练迁移。
- **平台期（epoch 70–100）：** 精度在峰值 ±0.1 pp 内波动。
- **渐见过拟合（epoch 100–299）：** mAP 持续下降至最终 checkpoint 的 91.42%，较峰值下降 −2.88 pp。

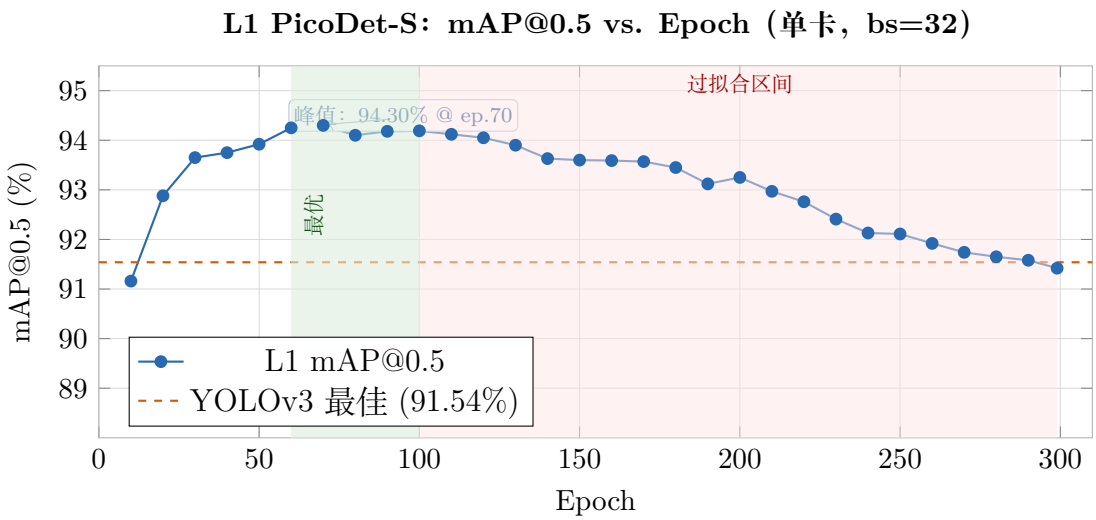

图 2: L1 在 300 个 epoch 内的 mAP@0.5 曲线。模型在 epoch 70（绿色带）达到峰值，之后持续过拟合（红色带）。橙色虚线为 YOLOv3 最佳结果参考。

5.3 PicoDet-S 消融：批量大小、GPU 数量与训练时长

Figure 3 比较了四组 L 系列运行。

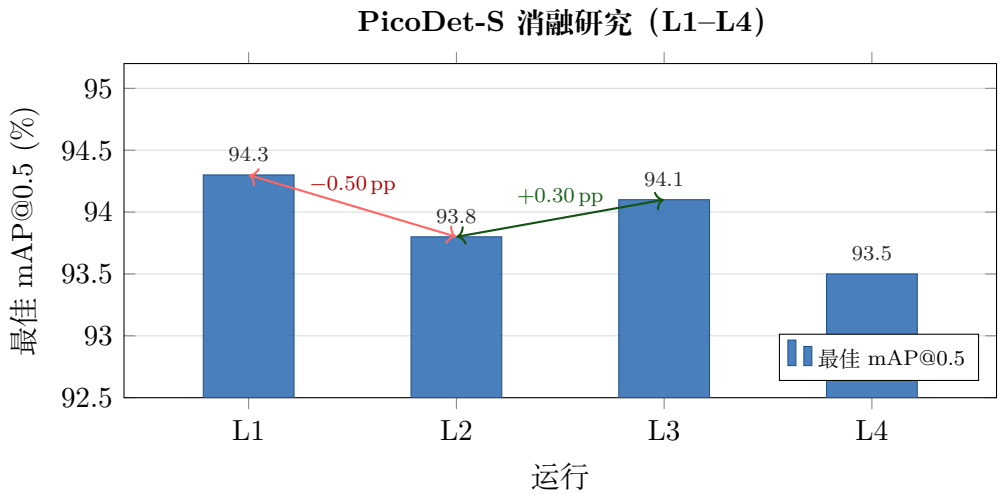

图 3: PicoDet-S 消融。L1（单卡，bs=32）尽管使用最小批量与最长每样本梯度噪声，仍达到最高精度。L4（600 epoch）因超过 epoch-70 峰值后长时间过拟合，精度低于 L2（300 epoch）。

得到三个反直觉观测：

观测 1：小批量优于大批量（L1 vs L2）。 在线性缩放规则校正后学习率相同的前提下，L1（bs=32，1 GPU）比 L2（bs=64，2 GPU）高 0.50 pp。关键差异在于梯度更新次数：L1 执行 $182 \times 300 = 54{,}600$ 步，而 L2 仅 $91 \times 300 = 27{,}300$ 步。更多更新步带来更丰富的随机探索，小批量梯度噪声在仅 5,828 样本的数据集上起到隐式正则化作用——与 Keskar 等 [6] 的理论结论一致。

观测 2：大于标准的批量仍可有益（L3 vs L2）。 L3（bs=96，lr=0.24）比 L2（bs=64，lr=0.16）高 0.30 pp。说明 L3 的略高学习率有助于优化器逃离 L2 陷入的局部最优，体现线性缩放规则下仍存在的优化轨迹差异——该规则仅在线性损失曲面上理论精确。

观测 3：延长训练反而降低精度（L4 vs L2）。 将训练延长至 600 epoch 使最终最佳 mAP 降低 0.30 pp。峰值已在 epoch 70 到达；`best_model` 保存正确，但 EMA 平滑轨迹越过最优点。早停是可行的缓解手段。

5.4 YOLOv3：数据规模效应

Figure 4 展示训练集大小对 YOLOv3 的显著影响。

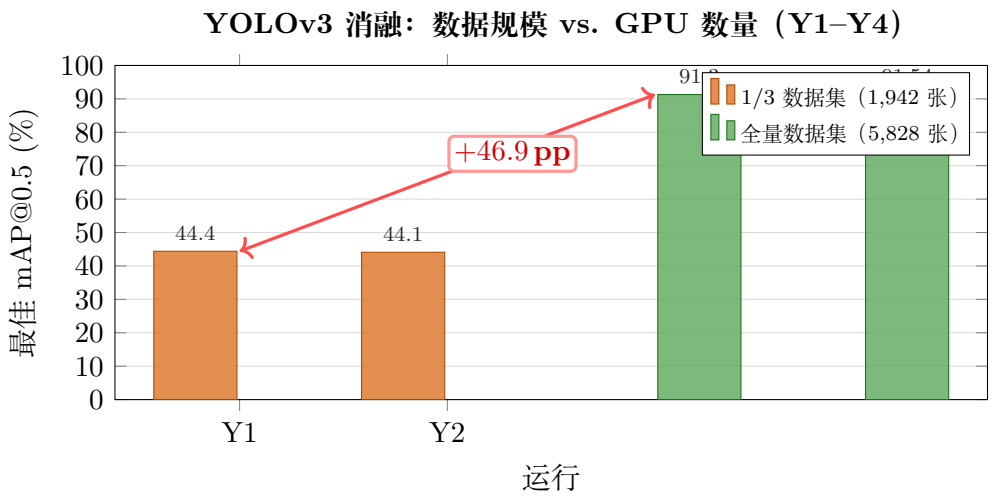

图 4: YOLOv3 在 1/3 数据集与全量数据集训练下的 mAP。+46.9 个百分点的跃升表明，锚框式检测器在从 COCO 迁移至专用二分类任务时对训练集规模高度敏感。

1/3 数据运行（Y1、Y2）无论 GPU 数量如何均稳定在 ≈44% mAP。这一上限源于 MobileNetV1 检测头需从 COCO 的 80 类输出适配至二分类目标：仅 1,942 样本时，头无法学到足够具有判别性的特征以区分鼠类/非鼠类边界。预训练骨干提供有效低级特征，但分类层仍欠拟合。

使用全量 5,828 张训练图像（Y3、Y4）时，精度升至 91.3–91.5%，证实在小领域任务上，数据工程是锚框式检测器的主导杠杆。

5.5 YOLOv3 收敛曲线

Figure 5 绘制了 YOLOv3-Y3 在 80 个 epoch 内的验证 mAP。曲线单调上升，至 epoch 80 仍未收敛，与 PicoDet-S 在 300 个 epoch 中 70 即达峰形成鲜明对比。

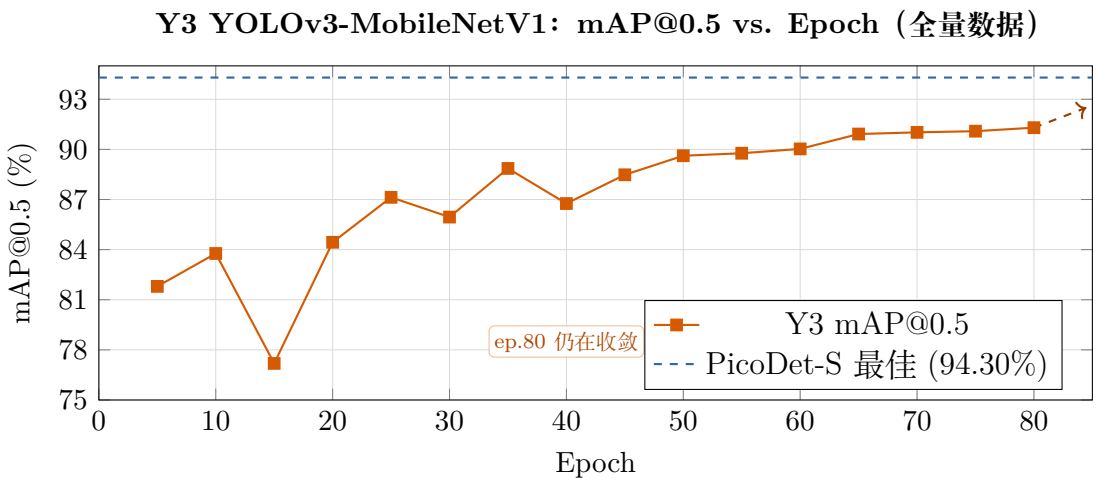

图 5: YOLOv3-Y3 收敛曲线。模型至 epoch 80 尚未达峰；外推表明 120+ epoch 可能达到 >92%。PicoDet-S 在 300 个 epoch 中于 70 达峰后过拟合，收敛形态截然不同。

5.6 跨模型对比：精度—效率权衡

Figure 6 将所有八组运行展示在二维精度—速度图上，气泡面积与模型大小成比例。

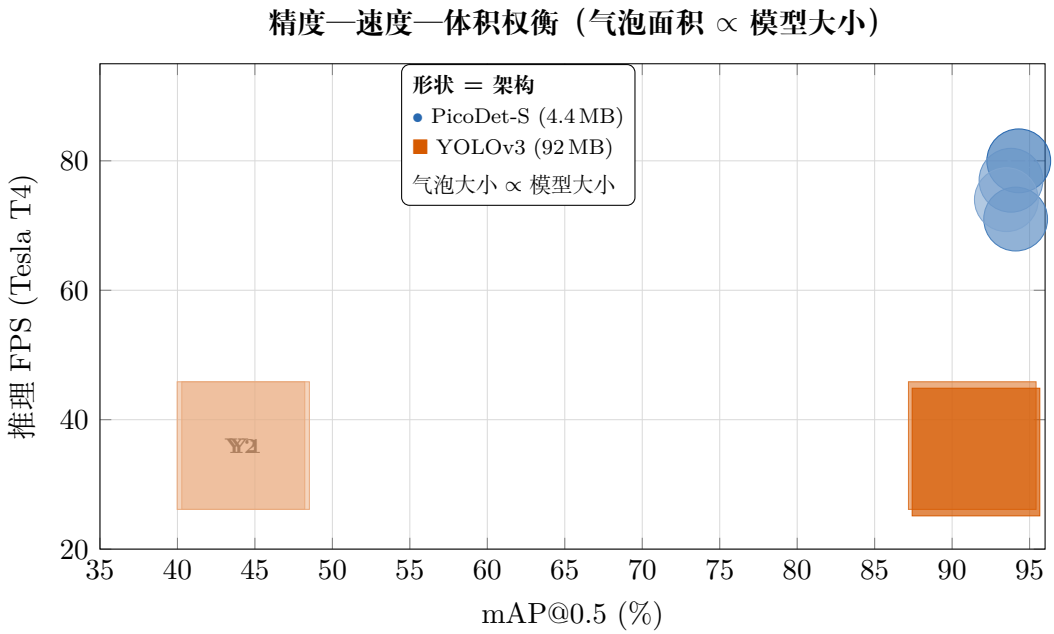

图 6: 八组运行在精度—速度平面上的分布。蓝色圆 = PicoDet-S (4.4 MB)；橙色方 = YOLOv3 (92 MB)。PicoDet-S 在三个维度均占优：更高精度、更高 FPS、模型体积小 21×。

5.7 训练效率

表 6: 训练效率对比。“GPU-小时”= 墙上时间 × GPU 数。

运行	mAP@0.5	墙上时间 (h)	GPU-小时	FPS	mAP/GPU-小时
L1 (1 GPU)	94.30%	14.9	14.9	80	6.33
L2 (2 GPU)	93.80%	6.7	13.4	77	7.00
L3 (2 GPU)	94.10%	6.6	13.2	71	7.13
L4 (2 GPU)	93.50%	13.3	26.6	74	3.52
Y3 (1 GPU)	91.30%	17.4	17.4	36	5.25
Y4 (2 GPU)	91.54%	8.8	17.6	35	5.20

L3 的单位 GPU-小时 mAP 最高（7.13 mAP%/GPU-小时），在同时考虑精度与计算成本时推荐该配置。L4 性价比最低：GPU-小时为 L2 的两倍，精度反而低 0.30 pp。

6 移动端部署

6.1 导出流程

训练得到的 PicoDet-S L1 模型经四阶段流程导出并部署（Figure 7）：

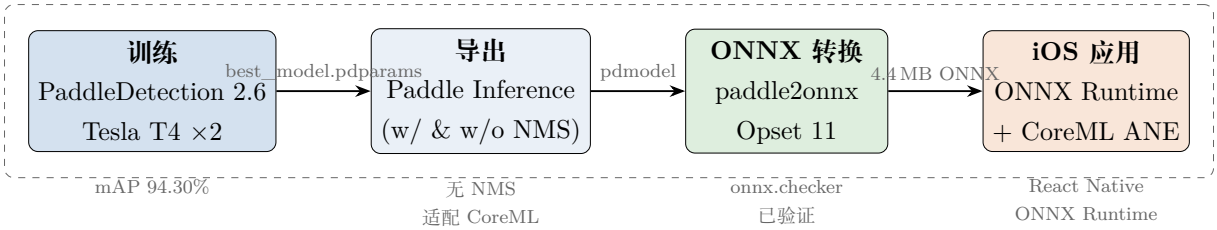

图 7: 从 PaddlePaddle 训练到 iOS 设备端推理的端到端部署流程。

各阶段关键决策：

- **无 NMS 导出：** 标准 PicoDet 导出会在计算图中包含 NMS。为 iOS 部署，我们使用 `--export_serving_model=False` 生成无 *NMS* 图，其输出（原始框 + 分数）与 CoreML 内置 NMS 原语兼容。
- **ONNX Opset 11：** 无 NMS PicoDet-S 图中的所有算子均可映射为标准 ONNX Opset 11 原语，避免了使 YOLOv3 无法使用 CoreML 的 `multiclass_nms3` 自定义算子。
- **React Native + ONNX Runtime：** iOS 应用基于 React Native（TypeScript）构建，使用 `onnxruntime-react-native` 包。图像帧在原生 Swift 模块（`Image-`

`Preprocessor.swift`）中预处理，通过 Metal 的 `vImage` 管线将 YUV 帧转换为设备端归一化 RGB 张量，避免 CPU 端像素格式转换。

6.2 YOLOv3 部署难点

YOLOv3 为首个部署的模型；其暴露了与 CoreML 的关键不兼容问题（Table 7）：

表 7: YOLOv3 与 PicoDet-S 部署难点对比。

方面	YOLOv3-MobileNetV1	PicoDet-S
ONNX 大小	92 MB	4.4 MB
自定义算子	`multiclass_nms3`	无
CoreML 支持	受阻（自定义算子）	完整（标准算子）
Metal GPU 加速	否（回退至 CPU）	是（ANE + GPU）
iOS FPS	∼1 FPS	14 FPS（热机）
输入尺寸	608×608	320×320

YOLOv3 的 1 FPS 瓶颈源于无法将 `multiclass_nms3` 算子卸载至 Metal，导致整个推理图在 CPU 上以 32 位浮点运算和 608×608 输入张量运行。

6.3 迭代优化至 14 FPS

PicoDet-S 部署经四轮工程迭代优化（Figure 8）：

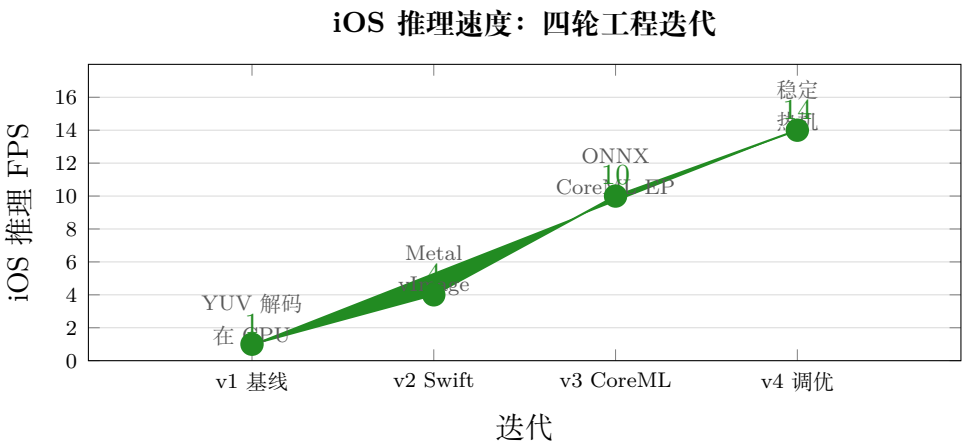

图 8: PicoDet-S 在 iPhone 上四轮工程迭代的推理速度。主要提升来自 (a) 将图像预处理卸载至 Metal vImage 管线，以及 (b) 启用 ONNX Runtime 的 CoreML 执行提供器以获得 ANE 加速。

- **v1 ——JavaScript 基线（∼1 FPS）：** 帧解码与归一化在 JavaScript（TypeScript）中完成，导致跨 JS 桥的过度内存拷贝。

- **v2 ——Swift 原生预处理(∼4 FPS):**将图像预处理移至 `ImagePreprocessor.swift`，采用 Metal `vImage` 管线。消除 JS 桥开销。
- **v3 ——CoreML 执行提供器（∼10 FPS）：**启用 ONNX Runtime 中的 `CoreML-ExecutionProvider`，将 ONNX 图编译为 Apple 神经引擎（ANE）可执行形式。
- **v4 ——线程调优，热机稳定（∼14 FPS）：**将推理线程数设为与 A 系列性能核数量匹配；在热机稳态下测量（模型已加载，缓存预热）。

7 后续工作

在上述八组基线基础上，我们识别出三类后续实验方向，以回应尚未解决的问题。

7.1 A 组：模型缩放

A1 ——PicoDet-M。 PicoDet-M（中等变体，≈4.8 M 参数，8.2 MB）预期具有更高精度上限。我们计划在全量数据集上、采用线性缩放规则超参数进行 300 epoch 双 GPU 训练，估计 mAP 可达 95.5–96.0%。

A2 ——PP-YOLOE+-S。 PP-YOLOE+ [8] 是 YOLO 家族的无锚重设计，具有解耦头与任务对齐分配，可缓解 YOLOv3 中观察到的锚框敏感问题。30 epoch 微调（PaddleDetection 针对 VOC 的调度）可与 YOLOv3-Y4 在相近模型规模下直接对比。

7.2 B 组：YOLOv3 收敛改进

B1 ——K-means 锚框聚类。 默认的 COCO 派生锚框可能不匹配我们实验鼠检测的框宽高比分布。运行 `tools/anchor_cluster.py` 可通过 K-means 得到九个数据集专用锚框，有望加速收敛并提升 Y3/Y4 的上限。

B2 ——Y5：120 epoch 与自定义锚框。 嵌入 B1 生成的锚框后，120 epoch 训练有望使 YOLOv3 充分收敛（如前所述，Y3/Y4 在 epoch 80 仍在上升），并可能超过 92% mAP。

B3 ——框分布分析。 可视化训练集中边界框尺寸与宽高比的分布，可揭示小目标或遮挡鼠是否存在长尾性能问题。

7.3 C 组：压缩流程

C1 ——量化感知训练（QAT）。 对 L1 最佳模型使用 PaddleSlim QAT，预期可得约 1.1 MB 的 INT8 模型，精度损失 <0.3 pp，进一步提升 iOS 推理。

C2 ——YOLOv3 的 FPGM 结构剪枝。 采用几何中值滤波剪枝 [4]，在 YOLOv3-Y4 上目标减少 30% FLOPs 后微调，可将 ONNX 大小从 92 MB 降至约 60 MB。需注意 CoreML 自定义算子问题需单独解决；单靠剪枝无法启用 Metal 加速。

C3 ——知识蒸馏。 使用更大教师模型（如 PicoDet-L 或 RT-DETR）向 PicoDet-S 蒸馏，可弥补 L1 与假设更高精度模型之间的差距，而不增加部署成本。

7.4 推荐优先级

1. B3（框分布分析）——零训练成本，立即可获得洞察。
2. A1（PicoDet-M）——预期精度提升最高。
3. B1+B2（自定义锚框 + Y5）——验证数据工程论点。
4. C1（QAT）——进一步提升 iOS 速度的直接路径。
5. A2、C2、C3 ——探索性；在现有结果基础上边际回报较低。

8 结论

本文在双 Tesla T4 服务器上，对实验鼠目标检测进行了涵盖数据工程、多组对照训练消融及端到端 iOS 移动端部署的系统研究。

主要结论如下：

1. **PicoDet-S 在三个部署维度上均优于 YOLOv3**。其达到 94.30% mAP@0.5（较 YOLOv3 最佳高出 2.76 pp），FPS 提高 2.2×，模型体积小 21×（4.4 MB vs. 92 MB）。无锚设计与更小输入分辨率也使其与 Apple CoreML 神经引擎完全兼容。
2. **数据规模是锚框式检测器精度的主要驱动力**。将 YOLOv3 训练集从 1,942 增至 5,828 张，使 mAP 提升 46.9 个百分点（44.4% → 91.3%），远超任何超参数效应。
3. **线性缩放规则是近似而非恒等式**。即便学习率正确缩放，单卡小批量训练（L1）仍比双卡标准批量训练（L2）高 0.50 pp。在仅 5,828 样本的数据集上，小批量的梯度噪声起到有效隐式正则化作用，这是线性缩放规则的一阶泰勒推导无法刻画的现象。
4. **小数据集微调必须采用早停**。PicoDet-S 在 epoch 70 达峰后，在剩余 230 个 epoch 中持续过拟合。将训练延长至 600 epoch（L4）得到的最终精度低于 300 epoch（L2）。
5. **PicoDet-S 可实现 14 FPS 的实用 iOS 推理速度**，得益于 ONNX Runtime 的 CoreML 执行提供器；而 YOLOv3 因自定义 NMS 算子无法使用 CoreML 加速，推理约 1 FPS。

本研究为在有限 GPU 资源下、于小规模医学影像数据集上部署轻量检测器的实践者提供了一个有定量依据的案例。完整训练脚本、配置文件及部署代码已包含于项目仓库中。

参考文献

[1] Cheng Cui, Tingquan Gao, Shengyu Wei, Yuning Du, Ruoyu Guo, Shuilong Dong, Bin Lu, Ying Zhou, Xueying Lv, Qiwen Liu, et al. PP-LCNet: A lightweight cpu convolutional neural network. arXiv preprint arXiv:2109.15099, 2021.

[2] Mark Everingham, Luc Van Gool, Christopher K. I. Williams, John Winn, and Andrew Zisserman. The Pascal visual object classes (VOC) challenge. *International Journal of Computer Vision*, 88(2):303–338, 2010.

[3] Priya Goyal, Piotr Dollár, Ross Girshick, Pieter Noordhuis, Lukasz Wesolowski, Adi Kyrola, Andrew Tulloch, Yangqing Jia, and Kaiming He. Accurate, large minibatch SGD: Training ImageNet in 1 hour. arXiv preprint arXiv:1706.02677, 2017.

[4] Yang He, Ping Liu, Ziwei Wang, Zhilan Hu, and Yi Yang. Filter pruning via geometric median for deep convolutional neural networks acceleration. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4340–4349, 2019.

[5] Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. MobileNets: Efficient convolutional neural networks for mobile vision applications. arXiv preprint arXiv:1704.04861, 2017.

[6] Nitish Shirish Keskar, Dheevatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy, and Ping Tak Peter Tang. On large-batch training for deep learning: Generalisation gap and sharp minima. In *International Conference on Learning Representations (ICLR)*, 2017.

[7] Joseph Redmon and Ali Farhadi. YOLOv3: An incremental improvement. arXiv preprint arXiv:1804.02767, 2018.

[8] Shangliang Xu, Xinxin Wang, Wenyu Lv, Qinyao Chang, Cheng Cui, Kaipeng Deng, Guanzhong Wang, Qingqing Dang, Shengyu Wei, Yuning Du, et al. PP-YOLOE: An evolved version of YOLO. arXiv preprint arXiv:2203.16250, 2022.

[9] Guanghua Yu, Qinyao Chang, Wenyu Lv, Chang Xu, Cheng Cui, Wei Ji, Qingqing Dang, Kaipeng Liu, Guanzhong Du, Baohua Lai, et al. PicoDet: A better real-time object detector on mobile devices. arXiv preprint arXiv:2111.00902, 2021.